MEMO Number: P04
DATE: 10/26/2026
TO: Kid Kidd
FROM: Paa Woode
SUBJECT: Project 4

1 INTRODUCTION

As engineers, sometimes there can be additions to projects. The purpose of this project 4 is to implement and document an addition to Project 3 (PROJECT-MYMILLI). This addition is adding a DC motor-driven fan that is driven by a PWM signal and has 4 states: OFF, LOW, MEDIUM, and HIGH. The speed of the fan is selected with a single push button that steps through the settings. The change as requested by Prof. Kidd in EC0 0.3.1 and is driven by additional customer requirements (Table 1). The existing Customer Requirements and High-level Technical Requirements for the LEDs and the serial dialog are unchanged, and new High-level Technical Requirements and Educational Constraints have been added. This document is a new revision of project 3. It includes an Engineering Change Note (ECN3.1) in Section 7 and marks all changes to project 3 with the date of the change

2 PROBLEM STATEMENT

There are 3 parts to this assignment, mirroring projects 1, 2 and 3. In Part I, the Customer Requirements C1 through C6 of project 1 and the High-level Technical Requirements HL.1 through HL.6 are unchanged and are incorporated by reference. The added Customer Requirements, High-level Technical Requirements (HL.7 through HL.18) and Educational Constraints of ECO 0.3.1 are listed. In Part II, the testing requirements of projects 1, 2 and 3 (T.1 through T.25) are retained and extended with the tests needed to evaluate the fan and button (T.26 through T.42). In Part III, the system is redesigned so that a PWM-driven fan is controlled by a debounced push button task added to the cyclic executive. The system is then built and tested.

3 EQUIPMENT

- Arduino Uno R3 development board or clone, e.g. the ELEGOO Uno R3
- LEDs, qty 2 (unchanged)
- 330Ω ¼ Watt resistors, qty 2 (used in place of 200Ω, see project 1)
- 10kΩ ¼ Watt resistor, qty 1 (now used as the push button pull-down)
- Arduino IDE 2.3.3 or better, a computer, and a USB 2.0 type A to type B cable
- ADDED: small DC motor fan, operating voltage 3 V to 12 V, rated current less than 4 A, no larger than 4” x 4” x 3” (not including cage), qty 1
- ADDED: NPN transistor, 2N2222 (or equivalent logic-level switch), qty 1
- ADDED: 1kΩ ¼ Watt resistor (transistor base resistor), qty 1
- ADDED: 1N4001 diode (flyback diode across the fan), qty 1
- ADDED: single-pole, momentary-on, normally-off push button, qty 1

- ADDED: breadboard jumper wires

4 PART I: CUSTOMER REQUIREMENTS AND HIGH-LEVEL ENGINEERING REQUIREMENTS

PROBLEM TO BE SOLVED: The Customer Requirements C1 through C6 of project 1, Table 1, and the High-level Technical Requirements HL.1 through HL.6, project 1 Table 2, are unchanged. Table 1 lists the Customer Requirements added by ECO 0.3.1. They are for reference only. The engineering work is to implement the added High-level Technical Requirements in Table 2 and the Educational Constraints in Table 3.

Table 1. Added Customer Requirements (new, ECO 0.3.1). The label C6 is also used by project 1 for the Arduino Uno R3 requirement. In this document the added fan requirement is written C6 (ECO 0.3.1).

C1.	The User will be able to choose the blink rate of the LED with a button.
C2.	The User will choose between 4 rates plus off.
C3.	The blink rates must be set by an operator prior to the system being started.
C4.	Permitted blink rates must be perceivable by the viewer.
C5.	The minimum blink rate must be no slower than once every 5 seconds.
C6 (ECO 0.3.1)	The device must have a motor-driven fan
C7	The RPM of the fan must be controllable via a push button
C8	The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH
C9	Each button must cycle the speeds from OFF to LOW, MEDIUM, HIGH, MEDIUM, LOW, OFF
C10	LOW must be able to move air with minimal sound and minimum feeling of flow
C11	MEDIUM must be able to move air with moderate sound and a moderate feeling of air flow
C12	HIGH must be able to move air quickly with moderate sound and a distinct feeling of air flow

Table 2. Added High-level Technical Requirements (new, 10/26/2026). HL.1 through HL.6 are unchanged. See project 1, Table 2.

Table 2. High Level technical requirements

HL.1	The System must use at least 2 LEDs
HL.2	The System must use a standard Arduino compatible development board (e.g. the provided ELEGOO Uno R3)
HL.3	The System must communicate with The User via the Arduino IDE serial-monitor port
HL.4	Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.
HL.5	The User must set the blink interval of the LEDs in msec
HL.6	The blink rate of each LED must be constant unless changed by The User
HL.7	The device must have a DC motor-driven fan with an operating voltage between 3V and 12V
HL.8	The fan must operate at a minimum between 1500 and 6500 RPM
HL.9	The fan must draw less than 4 Amps
HL.10	The fan must be driven by a PWM signal with a maximum voltage of 5VDC
HL.11	The fan must have a maximum dimension less than 4” x 4” x 3” (not including cage)
HL.12	The RPM of the motor must be controllable using a single-pole, momentary-on, normally-off push button.
HL.13	The operation of the button and motor must have no noticeable effect upon the operation of previously implemented I/O (e.g. serial monitor and LEDs)
HL.14	The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH
HL.15	Each hit of the button must cycle the speeds from OFF to LOW to MEDIUM to HIGH to MEDIUM to LOW to OFF
HL.16	LOW must be able to move air with a sound volume less than 35dB and airflow levels between ~~30 45~~ 0.30 to 0.45 CFM ± 0.05 CFM (modified 4/1/2030, CFM requirement corrected)
HL.17	MEDIUM must be able to move air with a sound volume less than 40dB and airflow levels between ~~45 70~~ 0.45 to 0.70 ± 0.05 CFM (modified 4/1/2030, CFM requirement corrected)
HL.18	HIGH must be able to move air with a sound volume less than 50dB and airflow levels between ~~70 100~~ 0.70 to 1.00 ± 0.05 CFM (modified 4/1/2030, CFM requirement corrected)

Table 3. Educational Constraints. EC.1 through EC.5 of project 1 and EC.1.0 through EC.1.3 of project 3 are unchanged. E.4 and E.5 below come from ECO 0.3.1. E.4 modifies EC.4 of project 1. E.5 is new and is separate from EC.5 (bare metal programming), which still applies.

E.4	The Evaluator must be able to successfully have The System perform the dialog in Table 5. (deleted 3/20/2030) The Evaluator must be able to successfully have The System perform the dialog in Table 4 and Table 5. (added 3/20/2030) (ECO text label: EC 1.2)

E.5	H.16, H.17, H.18 (HL.16, HL.17, HL.18 in this document) do not have to be explicitly measured since we do not have the necessary testing equipment. Just provide some evidence that these requirements have been satisfied through justified inference such as through calculations. (ECO text label: EC 1.3)

Table 4. Example Dialog (the user inputs are the LED number and the interval). Unchanged from project 1.

Serial port I/O	**Notes**
What LED? (1 or 2) 2	
What interval (in msec)? 600	LED2 starts blinking at an interval of 300ms on and 300ms off. LED1 is unaffected.
What LED? (1 or 2) 1	
What interval (in msec)? 1600	LED1 starts blinking at an interval of 800ms on and 800ms off. LED2 is unaffected.
What LED? (1 or 2)	Waiting for next LED# interval pair

Table 5. Example Button Presses (new, 10/26/2026).

Button	**Fan**
Initial state: Button un-pressed	Initial state: Fan off
Press	Fan on low
Press	Fan on medium
Press	Fan on high
Press	Fan on medium
Press	Fan on low
Press	Fan off
(repeat sequence above)	

5 PART II: DERIVE TESTING REQUREMENTS

PROBLEM TO BE SOLVED: Derive the testing requirements (labeled T.#) that allow the company to prove to the customer that The Company has satisfied the contract. Testing requirements T.1 through T.10 (including T.4.1 and T.9.1 through T.9.4) are unchanged. See project 1, Part II. T.11 through T.16 are unchanged. See project 2, Part II. T.17 through T.25 are unchanged. See project 3, Table 7. Table 6 lists the added testing requirements (new, 10/26/2026).

Table 6. Added testing requirements (new, 10/26/2026).

T.26	(HL.7) The Evaluator must inspect the fan label or datasheet and confirm that it is a DC motor fan with an operating voltage range inside 3V to 12V, and that the supply voltage used by The System is inside that range.
T.27	(HL.8) The Evaluator must inspect the fan datasheet, and may use a tachometer or strobe app, to confirm that the speed of the fan at LOW, MEDIUM and HIGH is between 1500 and 6500 RPM.
T.28	(HL.9) With the fan on HIGH, the Evaluator must measure the fan current with a multimeter in series with the fan, and confirm it is less than 4 Amps. The datasheet rated current is acceptable evidence if the meter is not available.
T.29	(HL.10) The Evaluator must inspect the source code and confirm the fan is driven from a PWM pin of the Arduino (pin 9) using analogWrite(). The Evaluator must measure the PWM pin with a multimeter or oscilloscope and confirm the peak voltage does not exceed 5VDC.
T.30	(HL.11) The Evaluator must measure the fan with a ruler or calipers and confirm that each dimension is within 4” x 4” x 3” (not including cage).
T.31	(HL.12) The Evaluator must inspect the push button, using the datasheet or a continuity test, and confirm it is single-pole, momentary-on and normally-off, and that it is the only control of the fan speed.
T.32	(HL.14) The Evaluator must confirm that the fan can be placed in four visibly and audibly distinct states: OFF (not turning), LOW, MEDIUM and HIGH.
T.33	(HL.15, Table 5) Starting with the fan OFF, the Evaluator must press the button 7 times and confirm that the fan goes through the states in Table 5, OFF to LOW to MEDIUM to HIGH to MEDIUM to LOW to OFF, and that the 7th press starts the sequence over by going to LOW.
T.33.1	After a power-up or reset of the Arduino, the fan must be OFF until the button is pressed.
T.34	The Evaluator must press and release the button 20 times, with at least 0.5 seconds between presses, and confirm that the fan steps exactly 20 times (no skipped and no extra steps from contact bounce).

T.35	The Evaluator must press and hold the button for 5 seconds and confirm the fan changes state only once, at the press, and does not change again while the button is held or when it is released.
T.36	(HL.13) With both LEDs blinking at two different intervals, the Evaluator must press the button at least 10 times and, using a video timestamp, confirm that both LEDs keep blinking within ±5% of the entered interval and do not pause (compare with T.16).
T.37	(HL.13) With the fan stepping through its states, the Evaluator must enter several LED/interval pairs in the serial monitor and confirm that every prompt and response is correct and the serial monitor remains responsive.
T.38	The Evaluator must inspect the source code and confirm that the button/fan function is a task, taskButton(), in the function pointer table dispatched from loop(), that it contains no blocking call (delay(), busy-wait loop), that all its timing comes from mymilli(), and that neither millis() nor delay() is called anywhere (see T.12 and T.17).
T.39	(HL.16, HL.17, HL.18, E.5) The Evaluator must review the calculations in Part III, Tables 10 and 11, and confirm they show, from the fan datasheet and the PWM duty cycles, that LOW (T.39.1), MEDIUM (T.39.2) and HIGH (T.39.3) meet the sound and airflow limits of HL.16, HL.17 and HL.18 and the RPM limits of HL.8.
T.40	(E.4) The Evaluator must perform the dialog in Table 4 and the button presses in Table 5 in the same session and confirm The System does both with the results given in the tables.
T.41	(Regression) The Evaluator must re-execute T.1 through T.25 (see projects 1, 2 and 3) on the PROJECT-ADDMOTOR build and confirm the results are identical.
T.42	The Evaluator must inspect the source code and confirm that Timer2 is used only by mymilli() and that the fan PWM uses a different timer (Timer1, pin 9), so the two do not conflict.

6 PART III: DESIGN, BUILD AND SYSTEM

Schematic:

The LED part of the schematic is unchanged from project 1 (LED 1 on Pin 2 and LED 2 on Pin 3, with 330Ω resistors). The schematic of that part is repeated in Figure 1. The fan and button circuit added by this project is shown in Figure 2.

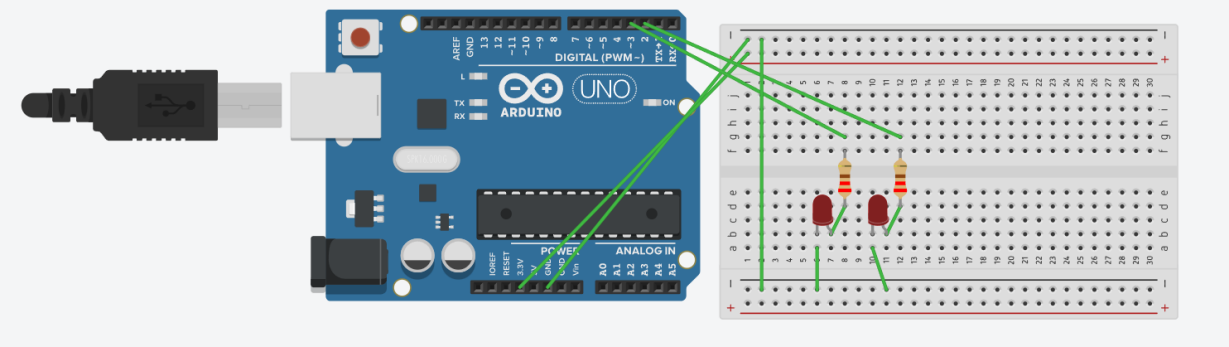


Figure 1

The LED part is unchanged from the previous projects, but there are new components now, like the motor, the NPN, and the diode.

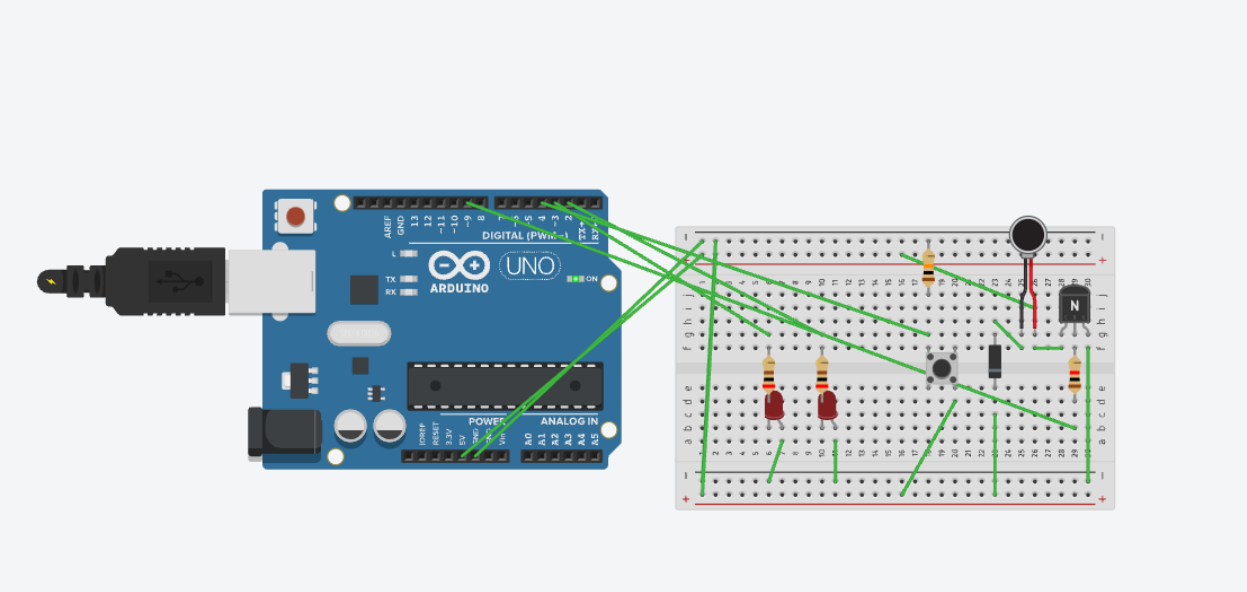


Figure 2

The fan is powered from the Arduino +5V pin. If a different fan is used whose current is larger than the Arduino can supply from the USB port (about 500 mA), the fan must have its own supply inside 3V to 12V with the grounds of the two supplies tied together, and the transistor must be rated for the fan current (HL.9, less than 4 Amps).

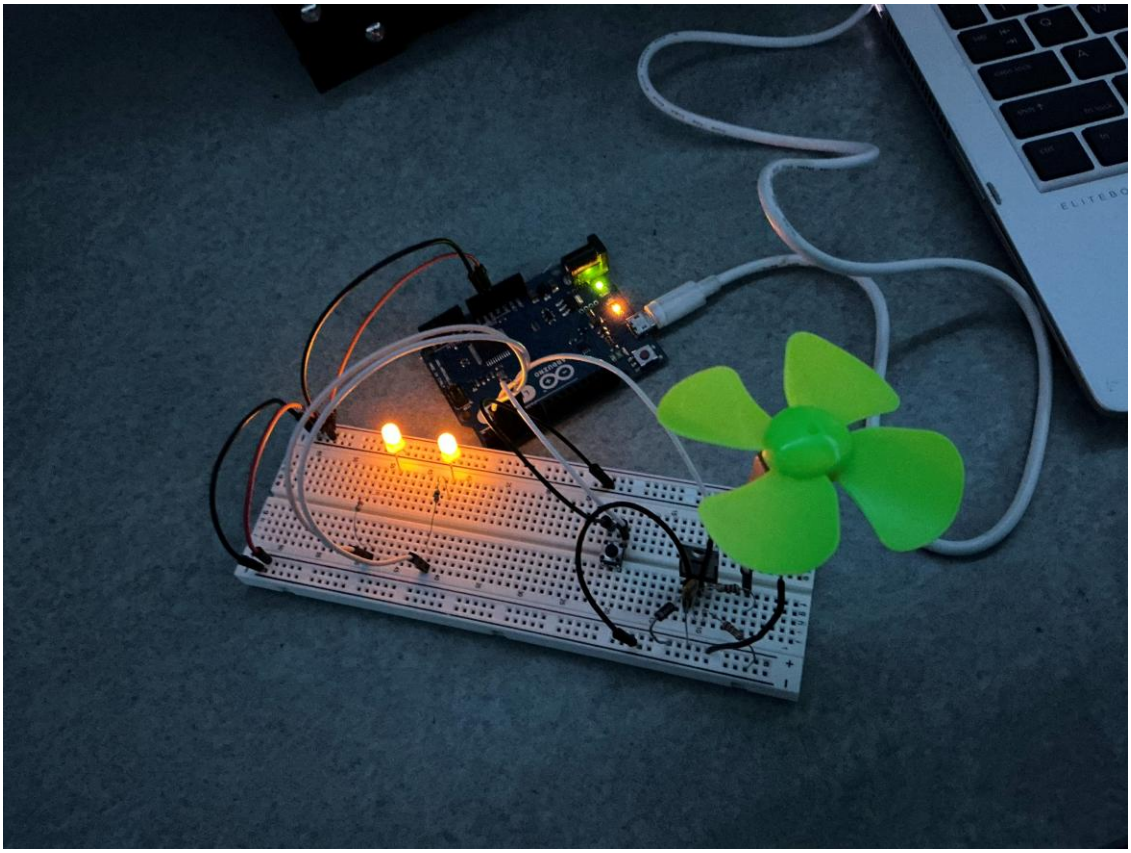


Figure 3

Software:

```
#include <stdint.h>
#include <avr/interrupt.h>
// Own register constants (EC.1.0.2) - unchanged from Project 3
#define MY_SREG    (*(volatile uint8_t*)0x5F)
#define MY_TIMSK2  (*(volatile uint8_t*)0x70)
#define MY_TCCR2A  (*(volatile uint8_t*)0xB0)
#define MY_TCCR2B  (*(volatile uint8_t*)0xB1)
#define MY_TCNT2   (*(volatile uint8_t*)0xB2)
#define MY_OCR2A   (*(volatile uint8_t*)0xB3)
#define MY_WGM21   1
#define MY_CS22    2
#define MY_OCIE2A  1
```

```
// mymilli(): timer, ISR, and reader - unchanged from Project 3
volatile uint32_t myTicks = 0;          // incremented every 1.000 ms

ISR(TIMER2_COMPA_vect) {
  myTicks++;
}

void myMilliInit() {
  MY_TCCR2A = (1 << MY_WGM21);          // CTC mode
  MY_TCCR2B = (1 << MY_CS22);           // prescaler 64 -> 250 kHz
  MY_OCR2A  = 249;                      // 250 counts = 1.000 ms
  MY_TCNT2  = 0;
  MY_TIMSK2 = (1 << MY_OCIE2A);         // compare-match A interrupt on
  __asm__ __volatile__("sei");
}

uint32_t mymilli() {
  uint8_t saved = MY_SREG;              // save interrupt state
  __asm__ __volatile__("cli");
  uint32_t copy = myTicks;              // atomic 32-bit read
  MY_SREG = saved;                      // restore interrupt state
  return copy;
}

// Cyclic executive (Project 2)
typedef void (*Task)(void);

//LED and serial dialog (unchanged from Project 3)
const int ledPin1 = 2;
const int ledPin2 = 3;
```

```
uint32_t interval1 = 0;
uint32_t interval2 = 0;
uint32_t lastToggle1 = 0;
uint32_t lastToggle2 = 0;
bool state1 = LOW;
bool state2 = LOW;

bool waitingForLed = true;
int  chosenLed = 0;
bool prompted = false;
char inBuf[12];
uint8_t inLen = 0;

// Fan and button (NEW, 10/26/2026)
const uint8_t buttonPin = 4;            // pressed = HIGH (10k pull-down)
const uint8_t motorPin  = 9;            // Timer1 PWM output (OC1A)

const uint32_t DEBOUNCE_MS = 30;

// Order the button walks through: OFF LOW MED HIGH MED LOW (then repeat)
const uint8_t SEQ_LEN = 6;
const uint8_t speedSeq[SEQ_LEN] = { 0, 1, 2, 3, 2, 1 };

// PWM duty (0-255) for level 0=OFF, 1=LOW, 2=MEDIUM, 3=HIGH
//   LOW  = 102/255 = 40.0 %
//   MED  = 166/255 = 65.1 %
//   HIGH = 255/255 = 100 %
const uint8_t dutyTable[4] = { 0, 102, 166, 255 };
```

```
uint8_t  seqIndex     = 0;              // start in OFF
bool     lastReading  = LOW;             // raw pin value last time
bool     stableState  = LOW;             // debounced button state
uint32_t lastChange   = 0;              // mymilli() of last raw change

void setMotorSpeed(uint8_t level) {
  analogWrite(motorPin, dutyTable[level]);
}

// print the status line after a button press
void printFanStatus(uint8_t level) {
  Serial.print(F("Button pressed -> Fan: "));
  switch (level) {
    case 0:  Serial.print(F("OFF"));    break;
    case 1:  Serial.print(F("LOW"));    break;
    case 2:  Serial.print(F("MEDIUM")); break;
    default: Serial.print(F("HIGH"));   break;
  }
  Serial.print(F(" (compare value "));
  Serial.print(dutyTable[level]);
  Serial.println(F(")"));
}

// Non-blocking line reader
bool readLine() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      if (inLen == 0) continue;
      inBuf[inLen] = '\0';
```

```
      inLen = 0;
      return true;
    }
    if (inLen < sizeof(inBuf) - 1) inBuf[inLen++] = c;
  }
  return false;
}

void blink(int pin, uint32_t interval, uint32_t &lastToggle, bool &state) {
  if (interval == 0) {
    if (state) { state = LOW; digitalWrite(pin, LOW); }
    return;
  }
  if (mymilli() - lastToggle >= interval) {
    state = !state;
    digitalWrite(pin, state);
    lastToggle += interval;
  }
}

// Tasks
void taskSerial() {
  if (!prompted) {
    Serial.println(waitingForLed ? F("What LED? (1 or 2)")
                                 : F("What interval (in msec)?"));
    prompted = true;
  }
  if (!readLine()) return;

  long v = atol(inBuf);
```

```
  if (waitingForLed) {
    if (v == 1 || v == 2) {
      chosenLed = v;
      waitingForLed = false;
    } else {
      Serial.println(F("Enter 1 or 2."));
    }
  } else {
    uint32_t half = (v > 1) ? v / 2 : 0;
    if (chosenLed == 1) { interval1 = half; lastToggle1 = mymilli(); }
    else                { interval2 = half; lastToggle2 = mymilli(); }
    waitingForLed = true;
  }
  prompted = false;                       // re-prompt on the next pass
}

void taskLed1() { blink(ledPin1, interval1, lastToggle1, state1); }
void taskLed2() { blink(ledPin2, interval2, lastToggle2, state2); }

// Debounce the button; one press = one step through speedSeq[].
void taskButton() {
  bool reading = digitalRead(buttonPin);

  if (reading != lastReading) {          // raw pin changed: restart timer
    lastReading = reading;
    lastChange  = mymilli();
  }

  if ((mymilli() - lastChange >= DEBOUNCE_MS) && (reading != stableState)) {
    stableState = reading;               // change survived the debounce time
```

```
    if (stableState == HIGH) {          // act on the press, not the release
      seqIndex = (seqIndex + 1) % SEQ_LEN;
      setMotorSpeed(speedSeq[seqIndex]);
      printFanStatus(speedSeq[seqIndex]);
    }
  }
}

Task taskTable[] = { taskSerial, taskLed1, taskLed2, taskButton };
const uint8_t NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]);

void setup() {
  Serial.begin(9600);
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(buttonPin, INPUT);            // external 10k pull-down
  pinMode(motorPin, OUTPUT);
  setMotorSpeed(0);                     // fan OFF at power-up
  myMilliInit();
}

void loop() {
  for (uint8_t i = 0; i < NUM_TASKS; i++) taskTable[i]();
}
```

Table 12. Test results.

Test	**Origin**	**Pass / Fail**	**Notes / measured values**
T.1 - T.10	Retained, project 1		
T.11 - T.16	Retained, project 2		

Test	Origin	Pass / Fail	Notes / measured values
T.17 - T.25	Retained, project 3		
T.26	New		
T.27	New		
T.28	New		
T.29	New		
T.30	New		
T.31	New		
T.32	New		
T.33	New		
T.33.1	New		
T.34	New		
T.35	New		
T.36	New		
T.37	New		
T.38	New		
T.39	New		
T.40	New		
T.41	New (regression of T.1 - T.25)		
T.42	New		

7 ENGINEERING CHANGE NOTE

PROJECT	PROJECT-MYMILLI (Project #3)
ORIGINATION	Prof. Kidd, ECO 0.3.1 (3/1/2030, revised 4/1/2030)
PURPOSE OF CHANGE	Add a PWM-driven DC motor fan with 4 speeds controlled by one push button
DOCUMENT NUMBER	ECN3.1
DATE OF PUBLICATION	10/26/2026
LEAD ENGINEER	Kid Kidd, UMBC
TEAM LEAD	Paa Woode
EXECUTIVE SUMMARY	Per ECO 0.3.1, the design of project 3 was modified to add a DC motor fan driven by a PWM signal on Pin 9 through an NPN transistor. A momentary push button on Pin 4 steps the fan through OFF, LOW, MEDIUM, HIGH, MEDIUM, LOW, OFF. The button is read by a new non-blocking task, taskButton(), added to the function pointer table of the cyclic executive, with a 30 msec debounce timed by mymilli(). Customer requirements C1 through C6 and high-level requirements HL.1 through HL.6 are unchanged. Customer requirements C6 (ECO 0.3.1) through C12, high-level requirements HL.7 through HL.18 and educational constraints E.4 and E.5 are added. Testing requirements T.1 through T.25 are retained and T.26 through T.42 are added. Hardware is changed by the addition of the fan driver and the button.
HIGH-LEVEL DESCRIPTION	A PWM-driven fan and a push button are added to the design of project 3. The LED, serial and mymilli() designs are not changed.
REASON FOR CHANGE	The customer has added the requirement of having a cooling fan added to the device (ECO 0.3.1).

CHANGED DOCUMENTS	10/26/2026, project 3 Section 3, Equipment: fan, transistor, 1kΩ resistor, diode and push button added. The 10kΩ resistor is now used as the button pull-down. 10/26/2026, project 3 Section 4: Customer Requirements C6 (ECO 0.3.1) through C12, High-level Requirements HL.7 through HL.18 and Educational Constraints E.4 and E.5 added. EC.4: dialog in Table 4 E.4: dialog in Table 4 and Table 5. HL.16 through HL.18 use the CFM values corrected on 4/1/2030. 10/26/2026, project 3 Section 5: T.26 through T.42 added. 10/26/2026, project 3 Section 6, Hardware Design: Hardware: Unchanged. fan driver circuit and push button added (Figure 2, Table 7). 10/26/2026, project 3 Section 6, Software: taskTable[] = { taskSerial, taskLed1, taskLed2 } taskTable[] = { taskSerial, taskLed1, taskLed2, taskButton }. 10/26/2026, project 3 Section 7: "Testing requirements T.1 through T.16 were retained and T.17 through T.25 were added." revised to cover T.1 through T.42 (see Section 8 of this document).
ADDED DOCUMENTS	This revision (ECN3.1 included); source file ProjectAddMotor.ino with taskButton(), setMotorSpeed() and the speed tables; Figure 2 and Tables 7 through 12; tests T.26 through T.42.
REMOVED DOCUMENTS	None.
REPLACED DOCUMENTS	The hardware design and the task table of the software design of project 3 are replaced by Section 6 of this revision.
ANY ADDITIONAL DESIGN CONSIDERATIONS	Pin 9 (Timer1) drives the fan. Timer2 stays with mymilli(), so analogWrite() on Pins 3 and 11 is still unavailable. The fan must not be connected directly to an Arduino pin. The transistor and flyback diode are required. If the fan current exceeds what the USB port can supply, a separate supply inside 3V to 12V with a common ground is required (HL.7, HL.9). The duty values in dutyTable[] depend on the fan used and must be rechecked against its datasheet (Tables 9 through 11). The Arduino core is not modified. AVR gcc from Arduino IDE 2.3.3 or later (EC.1 of project 1). No 3rd-party multitasking libraries (EC.5 of project 1). LEDs remain on Pins 2 and 3. Resistors for the LEDs remain 330 Ω per project 1.
REPLACED DOCUMENTS	The hardware design and the task table of the software design of project 3 are replaced by Section 6 of this revision.
ANY ADDITIONAL DESIGN CONSIDERATIONS	Pin 9 (Timer1) drives the fan. Timer2 stays with mymilli(), so analogWrite() on Pins 3 and 11 is still unavailable. The fan must not be connected directly to an Arduino pin. The transistor and flyback diode are required. If the fan current exceeds what the USB port can supply, a separate supply inside 3V to 12V with a common ground is required (HL.7, HL.9). The duty values in dutyTable[] depend

	on the fan used and must be rechecked against its datasheet (Tables 9 through 11). The Arduino core is not modified. AVR gcc from Arduino IDE 2.3.3 or later (EC.1 of project 1). No 3rd-party multitasking libraries (EC.5 of project 1). LEDs remain on Pins 2 and 3. Resistors for the LEDs remain 330 Ω per project 1.

8 CONCLUSIONS

REFERENCE:

project 1: Reference document woode.cmpe311.project#1, Project 1 (PROJECT-ASYNC), 09/22/2026.

project 2: Reference document woode.cmpe311.project#2, Project 2 (PROJECT-CE), including ECN1.1, 09/29/2026.

project 3: Reference document woode.cmpe311.project#3, Project 3 (PROJECT-MYMILLI), including ECN2.1, 10/15/2026.

ECO 0.3.1: Engineering Change Order for Project 4 (PROJECT-ADDMOTOR), Prof. Kidd, 3/1/2030, revised 4/1/2030.